Raport weryfikacji

Naucz Pilota 1.0.0, 3 listopada 2026. Środowisko: macOS arm64, Node 24.11.0, npm 11.6.1; Chromium Playwright. Jest to raport techniczny lokalnego wykonania, nie badanie skuteczności nauczania.

Wyniki wykonane

- Silnik TypeScript: 24 rozwiązania witness dają SUCCESS; 37 knownFailures odrzucanych dokładnie oczekiwanym kodem. Przeniesiono 42 kontrole parsera i geometrii z walidatora Python, w tym dokładne skracanie dziesiętne, limity, cienką przeszkodę i wewnętrzny wierzchołek.
- Testy jednostkowe/integracyjne: 152 PASS w pięciu plikach Vitest. Obejmują silnik, maszynę stanów, 32 deterministyczne scenariusze INT z ai-evals i oficjalny SDK na lokalnej atrapie HTTP. Dziesięć wejść SEM sprawdzono strukturalnie; ich jakość semantyczna pozostaje NOT RUN.
- Python: PASS, 6 misji / 24 sceny / 24 witness / 37 błędów / 42 kontrole. Niezależny od TypeScript walidator referencyjny potwierdza dane, nie zastępuje testu aplikacji.
- Playwright: 7 PASS. Pełna kampania demo z 24 scenami; porażka robota w M04 nie blokuje gracza. Pierwszy wynik jest zachowany po odświeżeniu, poprawkach i powtórce. Przed human_check brak toru, robota, dziennika, wskazówek, obliczonych y i rozmowy.
- Interfejs: widoki 1366x768, 1440x900, 390x844; axe WCAG A/AA bez wykrytych naruszeń na starcie i practice; brak poziomego przewijania, klawiaturowa M01, tekst 200%, reduced motion i demo po wyłączeniu sieci. Zrzuty obejrzano wzrokowo. To ograniczony zakres, bez deklaracji pełnej dostępności dla wszystkich użytkowników.
- AI bez płatnego API: walidacja requestów i faz, ponowne obliczanie dowodów, odmowa, incomplete, deadline, maksymalnie jedna naprawa techniczna, jawna błędna propozycja, idempotencja, konflikt body/revision, limity, brak ukrytego demo i brak kontekstu przyszłych scen. Trwały licznik SQLite sprawdzony po zamknięciu i ponownym otwarciu bazy.
- Oficjalny SDK: testy HTTP completed, incomplete, refusal i 429; strict text.format, store:false, brak narzędzi i wyłączone automatyczne retry. To lokalna atrapa protokołu, nie prawdziwy model.
- Produkcyjny build: frontend bez source maps i bez danych witness/knownFailures. Statyczne walidatory zamiast runtime kompilacji Ajv. Serwer bundluje współdzielone pakiety do JavaScript.
- Electron macOS: cała M01 z lokalnych zasobów app://, bez backendu; izolacja kontekstu, sandbox, brak require/process w rendererze; PASS w checks/desktop-smoke.json.
- Windows x64: utworzono ZIP zawierający EXE PE i zasoby. Inspekcja ASAR wyklucza node_modules, .env, mapy źródeł i dane autora. Paczka niepodpisana. Cross-build jest osobną czynnością od testu uruchomienia.
- Nagrano rzeczywisty automatyczny przebieg M04 w trybie demo: MP4, 37,56 s, 1440x900, napisy objaśniające, bez udawania live i bez fikcyjnego głosu autora.

Uczciwe ograniczenia

Prawdziwe AI: NOT RUN. Brak OPENAI_API_KEY/AI_MODEL. Polecenie test:ai:live zapisuje powód i kończy się kodem 2, bez wywołań dostawcy. Nie oceniono jakości polskich odpowiedzi ani pokrycia live wszystkich umiejętności.

Windows runtime: NOT RUN. Nie ma dostępu do czystego komputera Windows. Paczkę należy rozpakować i przetestować niezależnie od środowiska autora. Dostępny jest tylko smoke macOS i inspekcja formatu PE/ASAR.

Hosting jury: NOT READY. distribution.json ma pusty origin HTTPS. Przygotowano Dockerfile, trwały magazyn i instrukcję, ale nie wykonano wdrożenia ani sprawdzenia z innego urządzenia. Dockerfile nie był budowany: docker info wykazało brak działającego demona i gniazda /Users/jakublewosz/.docker/run/docker.sock.

Pilotaż: NOT RUN. Nie uczestniczyły osoby testujące. Nie podano czasów użytkowników, ich liczby ani wzrostu wiedzy. Cel 30-45 minut pozostaje założeniem produktu.

Bezpieczeństwo zależności: npm audit --omit=dev wykazał 0 podatności produkcyjnych. Pełny npm audit wskazuje 25 moderate w narzędziach developerskich, związanych z minimatch i zależnymi narzędziami. Zwykłe npm audit fix ich nie usunęło; proponowane wymuszenie niekompatybilnego downgrade ESLint nie zostało zastosowane. Zależności te nie są pakowane do desktopu.

Formalności: nie uzupełniono danych i podpisów autora, nie zweryfikowano ponownie regulaminu przed wysyłką, nie opublikowano linku i nie wysłano zgłoszenia. Materiały mają status lokalnego pakietu do dalszego odbioru.

Naprawy ujawnione podczas testów

Naprawiono eksport fixture parsera, niezgodność runtime Ajv z CSP, eksporty statycznych walidatorów ESM, pozycję Piko przed lotem, pomijanie animacji, odblokowania po powtórce i nadmiarowe zależności pierwszego ASAR. Kontrola listy pól danych publicznych została rozszerzona o opcjonalne vertexLock z dalszych misji. Wyniki PASS dotyczą powtórzeń po tych naprawach.

Reprodukcja i dalszy odbiór

Uruchom npm ci; npm run typecheck; npm run lint; npm test; npm run test:content; npm run test:e2e; npm run build; npm run desktop:dist; npm run release:check. Test AI live jest osobny i może zużywać płatne API. Raporty maszynowe w checks/ podają rzeczywiste wyniki, a IMPLEMENTATION_STATUS.md określa końcowy stan etapów.

Po konfiguracji live wykonaj 10 scenariuszy semantycznych i teach/transfer sześciu umiejętności, przeczytaj odpowiedzi według rubryk, przetestuj publiczny endpoint na urządzeniu odbiorcy, następnie odbierz Windows i pilotaż. Nie łącz sukcesu robota z samodzielnym wynikiem gracza.